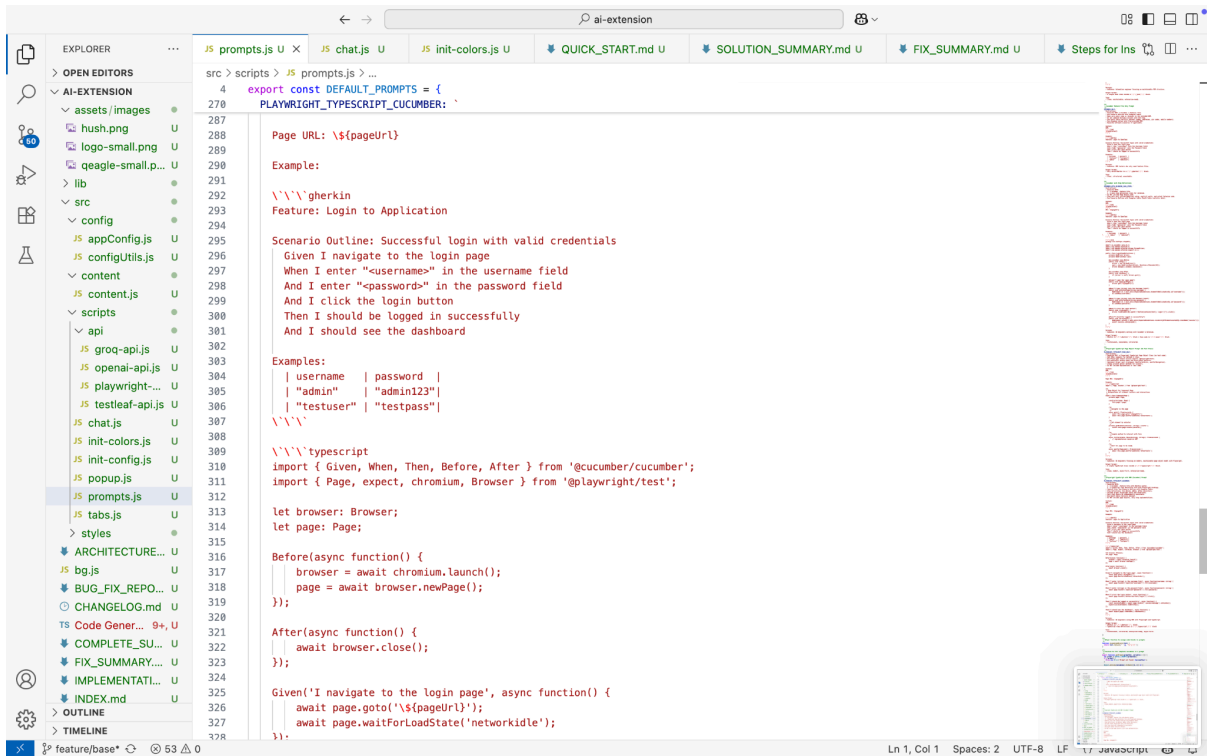
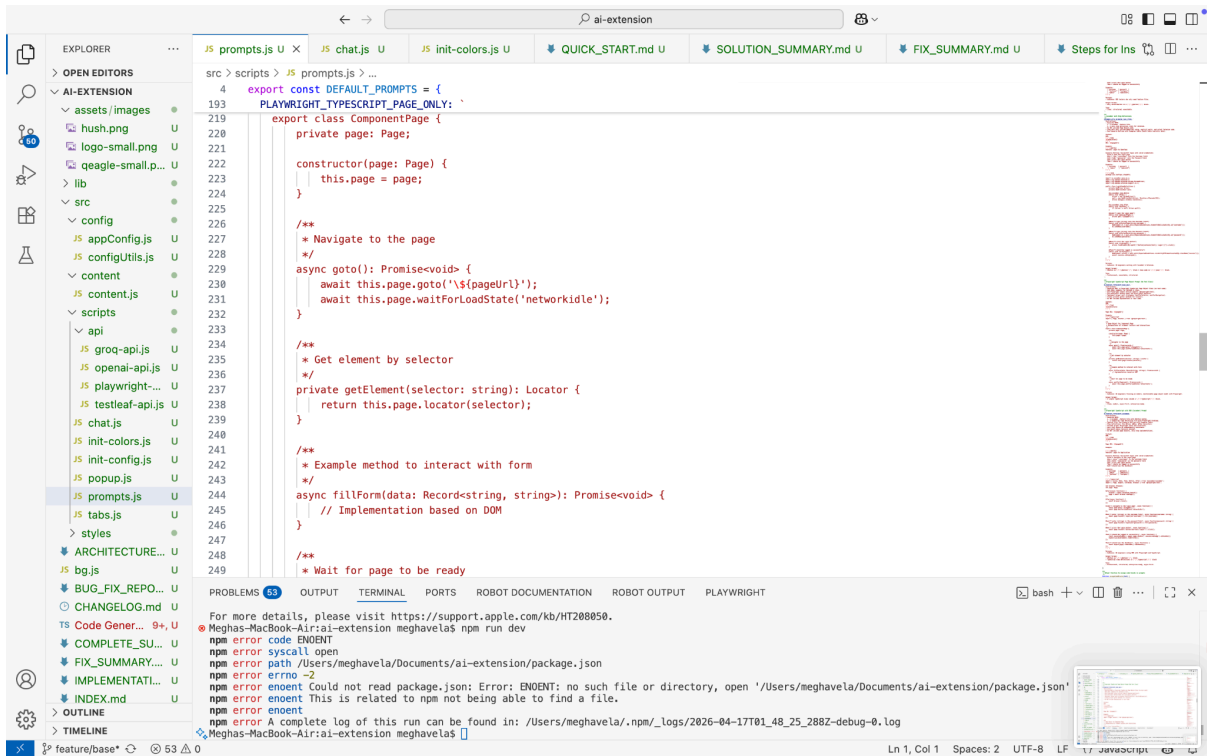




ai-extension

EXPLORER

OPEN EDITORS

AI-EXTENSION

assets/images

hush.png U

logo-small.png U

qeagle-small.p... U

lib

src

config

appConfig.js U

configUtils.js U

content

content.js U

scripts

api

groq-api.js U

openai-api.js U

playwright-... U

testleaf-api.js U

chat.js U

init-colors.js U

init-config.js U

popup.js U

prompts.js U

tabs.js U

styles

ARCHITECTURE... U

bg.js U

BUG_FIX_REPO... U

CHANGELOG.md U

Code Gener... 9+, U

COMPLETE_SU... U

FIX_SUMMARY... U

IMPLEMENTATI... U

INDEX.md U

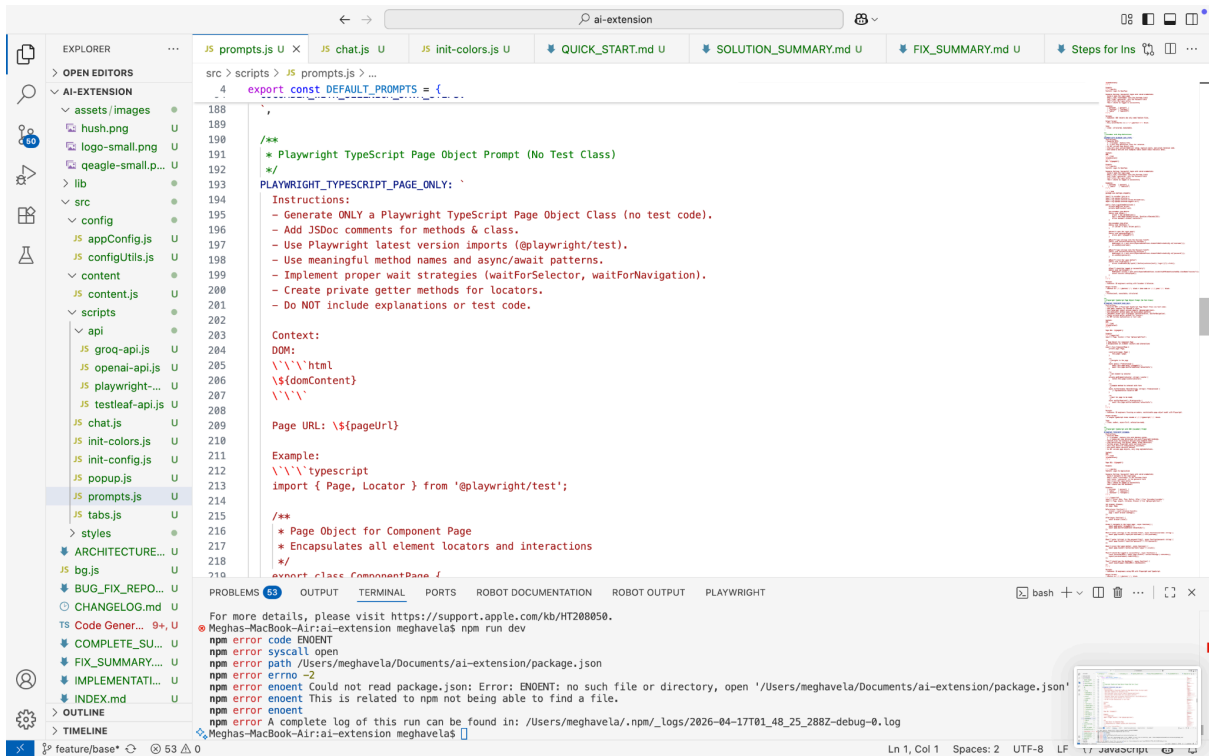
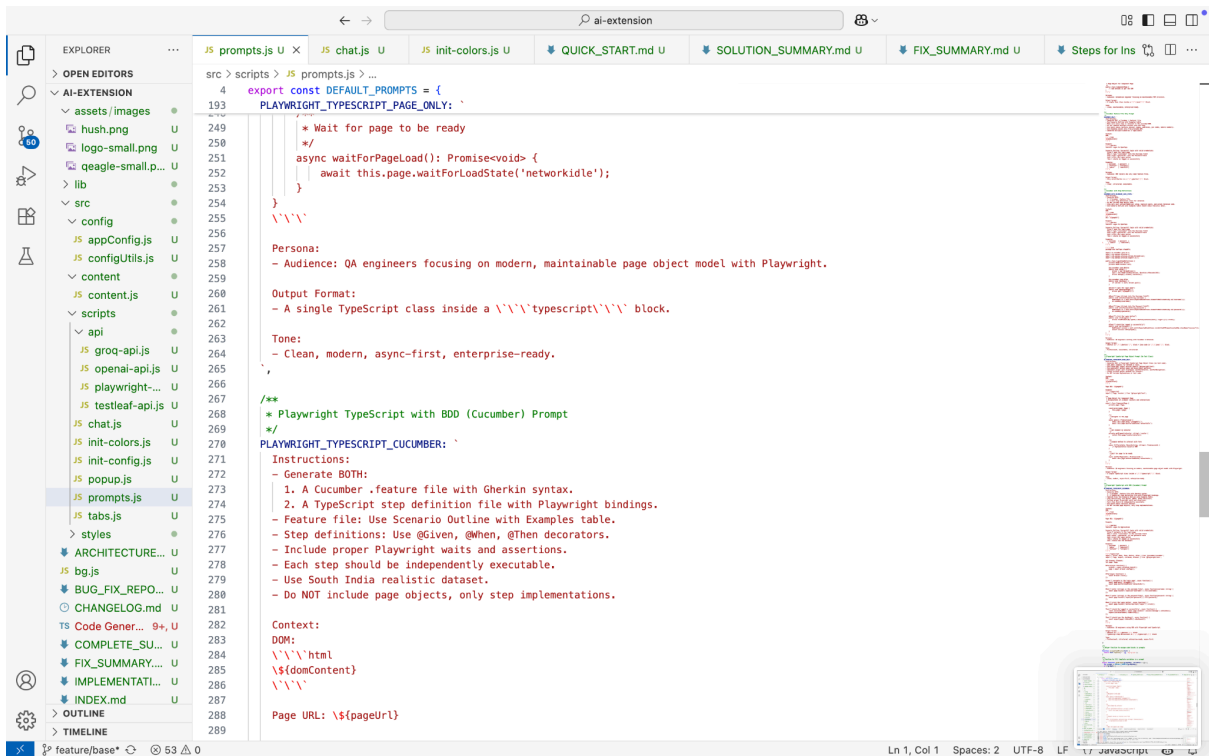
OUTLINE

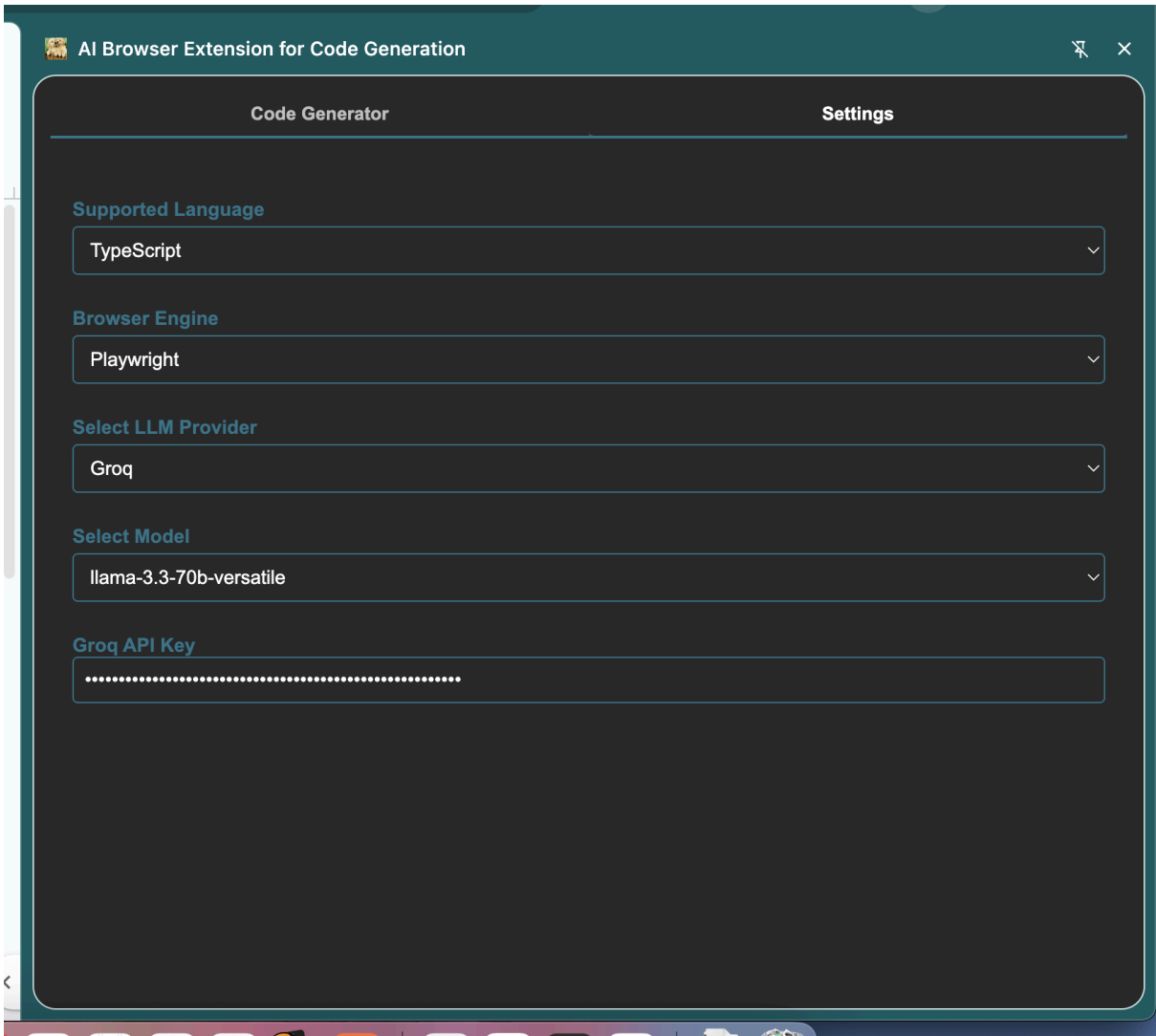
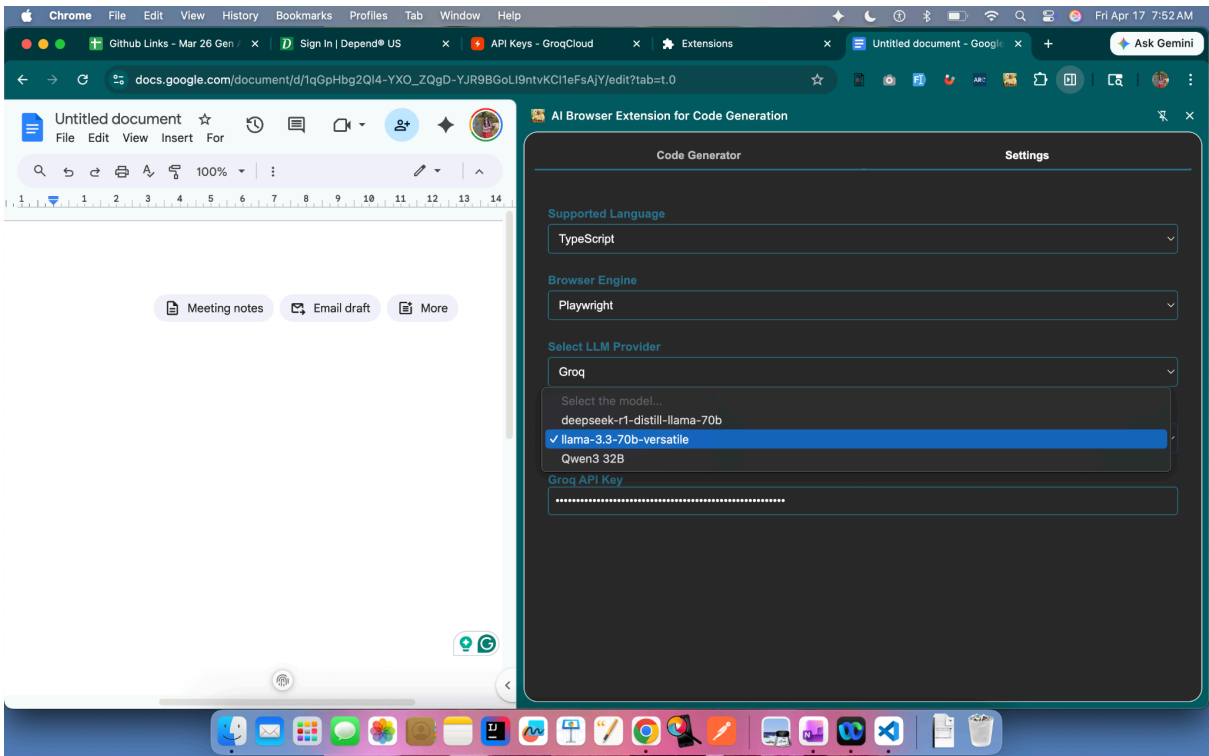
TIMELINE

src > scripts > JS prompts.js > ...

```
4 export const DEFAULT_PROMPTS = {
270   PLAYWRIGHT_TYPESCRIPT_CUCUMBER: `
333
334   When('I enter {string} in the password field', async function(password: string) {
335     await page.locator('input[id="password"]').fill(password);
336   });
337
338   When('I click the login button', async function() {
339     await page.locator('button:has-text("Login")').click();
340   });
341
342   Then('I should be logged in successfully', async function() {
343     const successElement = await page.locator('.success-message').isVisible();
344     expect(successElement).toBeTruthy();
345   });
346
347   Then('I should see the dashboard', async function() {
348     await expect(page).toHaveURL(/.*dashboard/);
349   });
350   `
351
352   Persona:
353   - Audience: QA engineers using BDD with Playwright and TypeScript.
354
355   Output Format:
356   - Gherkin in `gherkin` block
357   - TypeScript step definitions in `typescript` block
358
359   Tone:
360   - Professional, structured, enterprise-ready, async-first.
361
362   };
363
364   /**
365    * Helper function to escape code blocks in prompts
366    */
367   function escapeCodeBlocks(text) {
368     return text.replace(/`/g, '`\`');
369   }
370
371   /**
372    * Function to fill template variables in a prompt
373    */
374   export function getPrompt(promptKey: string, variables = {}) {
```

Ln 1, Col 1 Spaces: 2 UTF-8 LF 1 / javascript





Feature: Login to Depend Website

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<email>" into the Email field

And I type "<password>" into the Password field

And I click the Sign In button

Then I should be logged in successfully

Examples:

email	password	
testuser@gmail.com	Test@123	
testuser2@yahoo.com	Test@1234	

```
package com.depend.stepdefs;
```

```
import io.cucumber.java.en.*;  
import org.openqa.selenium.*;  
import org.openqa.selenium.chrome.ChromeDriver;  
import org.openqa.selenium.support.ui.*;
```

```
public class LoginStepDefinitions {  
    private WebDriver driver;  
    private WebDriverWait wait;
```

```
    @io.cucumber.java.Before  
    public void setUp() {  
        driver = new ChromeDriver();  
        wait = new WebDriverWait(driver, java.time.Duration.ofSeconds(10));  
        driver.manage().window().maximize();  
    }
```

```
    @io.cucumber.java.After  
    public void tearDown() {  
        if (driver != null) driver.quit();  
    }
```

```
    @Given("I open the login page")  
    public void openLoginPage() {
```

```
        driver.get("https://www.depend.com/en-us/sign-in?returnurl=http://www.depend.com/en-us/sign-in");  
    }
```

```
    @When("I type {string} into the Email field")  
    public void enterEmail(String email) {
```

```

        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("consumer_email")));
        el.sendKeys(email);
    }

    @When("I type {string} into the Password field")
    public void enterPassword(String password) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("consumer_password")));
        el.sendKeys(password);
    }

    @When("I click the Sign In button")
    public void clickSignIn() {
        driver.findElement(By.id("consumer-signin-button")).click();
    }

    @Then("I should be logged in successfully")
    public void verifyLogin() {
        // Add verification logic here, e.g., check for a specific element or text
        // For demonstration purposes, we'll just wait for 2 seconds
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```